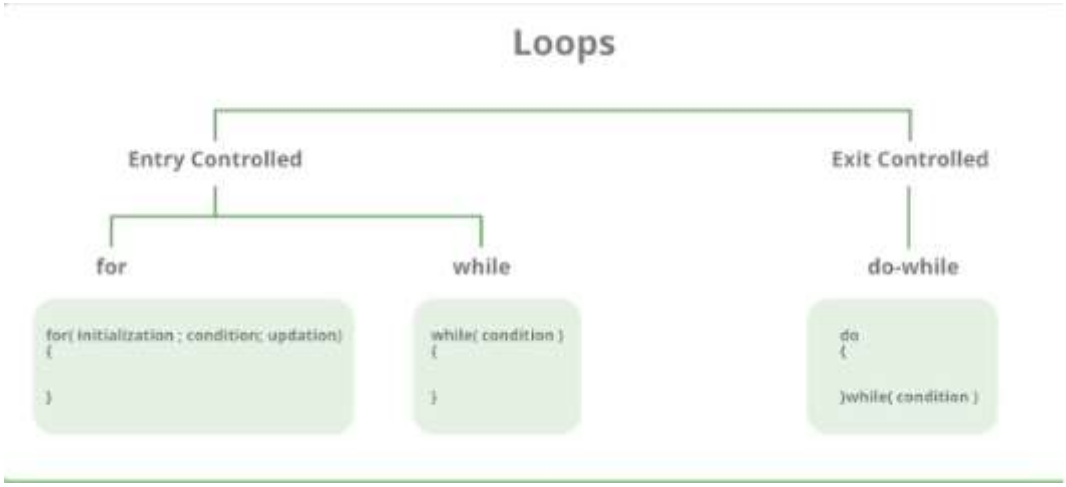
Looping constructs:-



1.Code for looping constructs.

```
#include <stdio.h>

int main() {
    int i;

    printf("\n--- FOR LOOP ---\n");
    // For loop
    for(i = 1; i <= 5; i++) {
        printf("%d ", i);
    }

    printf("\n\n--- WHILE LOOP ---\n");
    // While loop
    i = 1;
    while(i <= 5) {
        printf("%d ", i);
        i++;
    }

    printf("\n\n--- DO-WHILE LOOP ---\n");
    // Do-while loop
    i = 1;
    do {
        printf("%d ", i);
        i++;
    } while(i <= 5);
```

```
    return 0;
}
```

main.c

```c
    printf("\n\n--- WHILE LOOP ---\n");
    // While loop
    i = 1;
    while(i <= 5) {
        printf("%d ", i);
        i++;
    }

    printf("\n\n--- DO-WHILE LOOP ---\n");
    // Do-while loop
    i = 1;
    do {
        printf("%d ", i);
        i++;
    } while(i <= 5);

    return 0;
}
```

Output

```
--- FOR LOOP ---
1 2 3 4 5

--- WHILE LOOP ---
1 2 3 4 5

--- DO-WHILE LOOP ---
1 2 3 4 5

=== Code Execution Successful ===
```

Scope and lifetime of variables:-

Storage Specifier	Storage	Initial value	Scope	Life
auto	Stack	Garbage	Within block	End of block
extern	Data segment	Zero	global Multiple files	Till end of program
static	Data segment	Zero	Within block	Till end of program
register	CPU Register	Garbage	Within block	End of block

2.Code for scope and lifetime of variable.

```c
#include <stdio.h>

// Global variable (scope: entire program, lifetime: entire execution)
int globalVar = 100;

void function() {
    // Static variable (scope: local to function, lifetime: entire program)
    static int staticVar = 0;

    // Local variable (scope: within this function, lifetime: during function call)
    int localVar = 10;

    staticVar++;
```

```
    localVar++;

    printf("\nInside function:");
    printf("\nGlobal Variable = %d", globalVar);
    printf("\nStatic Variable = %d", staticVar);
    printf("\nLocal Variable = %d\n", localVar);
}

int main() {
    int i;

    printf("Global Variable in main = %d\n", globalVar);

    for(i = 1; i <= 3; i++) {
        function();
    }

    return 0;
}
```

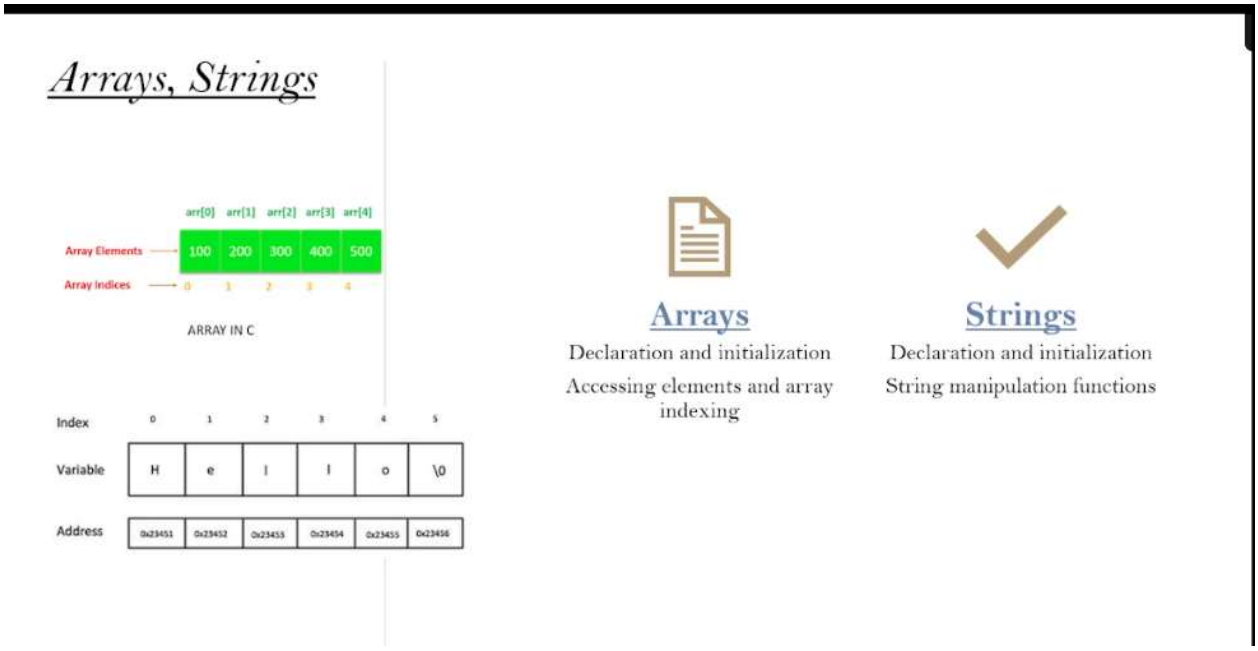
main.c | Share | Run

```
15
16      printf("\nInside function:");
17      printf("\nGlobal Variable = %d", globalVar);
18      printf("\nStatic Variable = %d", staticVar);
19      printf("\nLocal Variable = %d\n", localVar);
20  }
21
22  int main() {
23      int i;
24
25      printf("Global Variable in main = %d\n", globalVar);
26
27      for(i = 1; i <= 3; i++) {
28          function();
29      }
30
31      return 0;
32  }
```

Output | Clear

```
Global Variable in main = 100

Inside function:
Global Variable = 100
Static Variable = 1
Local Variable = 11

Inside function:
Global Variable = 100
Static Variable = 2
Local Variable = 11

Inside function:
Global Variable = 100
Static Variable = 3
Local Variable = 11
```

Arrays and strings:-

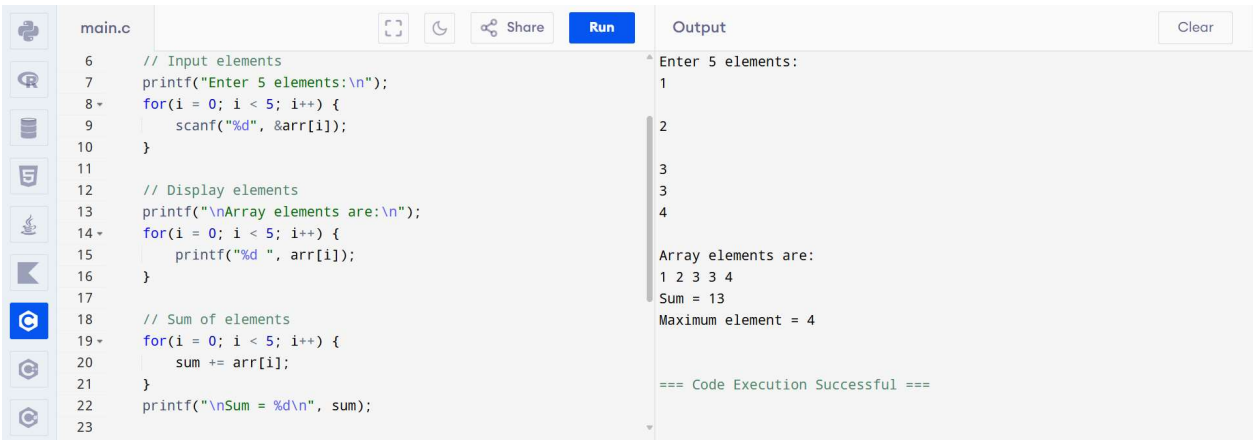


An array is a **collection of elements of the same data type stored in contiguous memory locations.**

3.**C program for Arrays** covering **declaration, initialization, input, and operations**:

```
#include <stdio.h>

int main() {
    int arr[5], i, sum = 0;

    // Input elements
    printf("Enter 5 elements:\n");
    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    // Display elements
    printf("\nArray elements are:\n");
    for(i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }

    // Sum of elements
    for(i = 0; i < 5; i++) {
        sum += arr[i];
    }
    printf("\nSum = %d\n", sum);
```

```
    // Find maximum element
    int max = arr[0];
    for(i = 1; i < 5; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
    }
    printf("Maximum element = %d\n", max);

    return 0;
}
```



A string in C is an **array of characters ending with a null character (`'\0'`).**

4.**C program for Strings** covering **input, output, and common operations**:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100], str2[100];

    // Input string
    printf("Enter a string: ");
    scanf("%s", str);

    // Display string
    printf("\nYou entered: %s\n", str);

    // Length of string
    printf("Length of string = %lu\n", strlen(str));

    // Copy string
```

```
    strcpy(str2, str);
    printf("Copied string = %s\n", str2);

    // Concatenate strings
    strcat(str, " World");
    printf("After concatenation = %s\n", str);

    // Compare strings
    if(strcmp(str, str2) == 0)
        printf("Strings are equal\n");
    else
        printf("Strings are not equal\n");

    return 0;
}
```

main.c Share Run Output Clear

```
    printf("Length of string = %lu\n", strlen(str));

    // Copy string
    strcpy(str2, str);
    printf("Copied string = %s\n", str2);

    // Concatenate strings
    strcat(str, " World");
    printf("After concatenation = %s\n", str);

    // Compare strings
    if(strcmp(str, str2) == 0)
        printf("Strings are equal\n");
    else
        printf("Strings are not equal\n");

    return 0;
}
```

```
Enter a string: abcdw

You entered: abcdw
Length of string = 5
Copied string = abcdw
After concatenation = abcdw World
Strings are not equal


=== Code Execution Successful ===
```